🎤 면접 예상 질문집 — 기술 면접 vs 프로젝트 면접

> 총 30문항. 각 질문에 **난이도**, **출제 빈도**, **모범 답변**, **꼬리 질문**을 포함.

Part A. 기술 면접 (React / Next.js / 웹 개발 일반)

기술 면접은 **프로젝트와 무관하게** React/웹 개발 지식을 묻는 질문.

A-1. React 핵심 개념

Q1. Virtual DOM이 뭐고, 왜 쓰나요? ⭐⭐⭐ (높은 빈도)

> **모범 답변**: 실제 DOM을 직접 조작하면 느리니까, 메모리에 가상의 DOM 트리를 만들어 두고 상태가 바뀌면 **이전 VDOM과 새 VDOM을 비교(diffing)**해서 **바뀐 부분만** 실제 DOM에 반영합니다. 이걸 **재조정(Reconciliation)**이라 합니다.

꼬리 질문:

- "key를 왜 써야 하나요?" → 리스트에서 어떤 아이템이 추가/삭제/이동됐는지 VDOM이 식별하려면 고유 key가 필요합니다. 배열 index를 쓰면 순서 변경 시 전체를 다시 렌더링합니다.
- "React 19에서 달라진 점은?" → 아래 상세 설명 참조.

> [!NOTE] **React 19에서 달라진 점 — 상세 설명**
>
> React 19는 2024년 말에 출시된 메이저 버전이다. 핵심 변화:
>
> **1. Concurrent 렌더링(동시 렌더링)이 기본값**
>
> - 이전: React가 한 렌더링을 시작하면 끝날 때까지 멈출 수 없었음 (화면 버벅임)
> - React 19: 렌더링 도중에도 "더 급한 작업"(사용자 타이핑 등)이 들어오면 **중간에 멈추고 급한 것부터 처리**. 이걸 "Concurrent(동시) 렌더링"이라 함
> - 비유: 요리하다가 손님이 오면 요리 잠시 멈추고 문 열어주는 느낌
>
> **2. Server Components (서버 컴포넌트)**
>
> - 컴포넌트 코드가 **서버에서만 실행**되고, 결과 HTML만 브라우저로 전송
> - 장점: 번들 크기 감소 (8KB 라이브러리를 import해도 사용자 다운로드 0KB), DB 직접 접근 가능
> - Svelte의 `+page.server.ts` 와 유사한 개념을 **컴포넌트 단위**로 적용
>
> **3. Server Actions (서버 액션)**
>
> - 서버에서 실행되는 함수를 클라이언트에서 **일반 함수처럼 호출** 가능
> - API Route를 안 만들어도 됨. `"use server"` 한 줄이면 서버 함수가 됨
> - 이 프로젝트의 CRUD 전부를 이걸로 처리함
>
> **4. `use()` hook 추가**
>
> - 기존에는 컴포넌트 안에서 Promise(비동기 작업의 결과값)를 직접 기다릴 수 없었음. `useEffect` + `useState` 조합으로 우회해야 했음
> - `use(promise)` 를 쓰면 비동기 데이터를 **변수처럼 바로 받을 수 있음**
>
> ```
> // 이전: 복잡한 3단계
> const [data, setData] = useState(null);
> useEffect(() => { fetchData().then(setData); }, []);
> if (!data) return <로딩/>;
>
> // React 19: 한 줄
> const data = use(fetchDataPromise); // Promise가 resolve될 때까지 Suspense가 로딩 표시
> ```
>
> **5. Suspense (서스펜스) 개선**
>
> - "Suspense" = "중지/보류"라는 뜻. 컴포넌트가 아직 준비 안 됐으면 **대신 로딩 UI를 보여줌**
> - `<Suspense fallback={<Skeleton />}>` 으로 감싸면, 안쪽 컴포넌트가 데이터를 기다리는 동안 `<Skeleton />` (뼈대 UI)이 표시됨
> - 이 프로젝트에서는 Next.js의 `loading.tsx` 파일이 자동으로 Suspense boundary 역할을 수행

Q2. useState와 useRef 차이는? ⭐⭐ (보통 빈도)

> **모범 답변**: `useState` 는 값이 바뀌면 **리렌더링을 유발**합니다. `useRef` 는 값이 바뀌어도 **리렌더링이 일어나지 않습니다**. `useRef` 는 DOM 요소 참조나, 렌더링에 영향을 주지 않는 값(타이머 ID, 이전 값 등)을 저장할 때 씁니다.

꼬리 질문:

- "useRef로 DOM을 직접 조작해도 되나요?" → 가능하지만, React의 선언적 패러다임을 깬다. 포커스, 스크롤, 측정 등 React가 제어할 수 없는 것에만 제한적으로.

> [!NOTE] **실무/면접 필수 React Hooks 정리**
>
> | Hook | 한 줄 설명 | 이 프로젝트에서 |
> |---|---|---|
> | `useState` | 상태값 저장 + 변경 시 리렌더링 | 폼 입력, 다이얼로그 열림/닫힘 |
> | `useEffect` | 컴포넌트 마운트/업데이트 시 부수효과 실행 | 다크모드 초기화, 타이머 |
> | `useRef` | 리렌더링 없이 값 보관 / DOM 참조 | 입력 필드 포커스 |
> | `useMemo` | 계산 비용이 큰 값을 캐싱(메모이제이션) | 필터링된 거래 목록 |
> | `useCallback` | 함수를 캐싱 (자식에게 전달 시 불필요한 리렌더 방지) | 이벤트 핸들러 |
> | `useContext` | Context(전역처럼 쓰는 데이터)를 읽기 | `ManualInputProvider` |
> | `useRouter` | Next.js 라우터 (페이지 이동) | 로그인 후 리다이렉트 |
> | `useTransition` | 급하지 않은 상태 업데이트를 "낮은 우선순위"로 표시 | 무거운 필터 변경 시 |

Q3. useEffect의 의존성 배열은 뭔가요? ⭐⭐⭐ (높은 빈도)

> **모범 답변**: 두 번째 인자인 배열에 넣은 값이 **변경될 때만** effect가 재실행됩니다. 빈 배열 `[]` 은 마운트 시 1번만. 배열을 안 넣으면 매 렌더마다 실행. cleanup 함수를 return하면 언마운트 또는 재실행 전에 호출됩니다.

```
useEffect(() => {
    const timer = setInterval(() => console.log("tick"), 1000);
    return () => clearInterval(timer); // cleanup
}, []); // 마운트 시 1번
```

꼬리 질문: "의존성을 빠뜨리면?" → stale closure 문제. 오래된 값을 참조하게 됨.

Q4. 클라이언트 상태 관리를 어떻게 하나요? ⭐⭐ (보통 빈도)

> **모범 답변**: 소규모 앱에서는 `useState` + props로 충분합니다. 깊은 전달이 필요하면 `Context API`. 대규모에서는 Redux, Zustand, Jotai 같은 외부 라이브러리를 씁니다. **이 프로젝트는 Server Component가 데이터를 직접 가져오므로 전역 상태 관리 라이브러리를 쓰지 않았습니다.**

꼬리 질문: "왜 Redux/Zustand를 안 썼나요?" → 아래 상세 설명 참조.

> [!NOTE] **서버 컴포넌트(Server Component)란?**
>
> 기존 React는 **모든 컴포넌트가 브라우저에서 실행**됐다. 서버 컴포넌트는 **서버에서만 실행되는 컴포넌트**다.
>
> ```
> [기존 React]
> 서버 → HTML 뼈대 전송 → 브라우저에서 JS 다운로드 → 브라우저에서 렌더링 + 데이터 fetch
>
> [Server Component]
> 서버에서 컴포넌트 실행 + DB 조회 → 완성된 HTML 전송 → 브라우저는 보여주기만
> ```
>
> **이 프로젝트에서의 활용**: 거래 목록 페이지(`/transactions`)를 열면, 서버에서 DB에 직접 접속해서 거래 데이터를 가져오고, 완성된 HTML을 보내줌. 브라우저는 `fetch()` 를 호출할 필요가 없음.
>
> **Server Action(서버 액션)이란?**
>
> 서버에서 실행되는 함수를 **클라이언트에서 직접 호출**할 수 있게 해주는 기능이다.
>
> ```
> // 서버 액션 파일 (src/server/actions/transaction.ts)
> "use server";  // ← 이 한 줄이 "이 파일의 함수는 서버에서 실행됨"을 선언
>
> export async function createTransaction(data: FormData) {
>  // 이 코드는 브라우저가 아니라 서버에서 실행됨
>  await db.insert(transactions).values({ ... });
>  revalidatePath("/transactions"); // 해당 페이지 캐시 무효화 → 자동 새로고침
> }
> ```
>
> ```
> // 클라이언트 컴포넌트 (브라우저에서 실행)
> "use client";
> import { createTransaction } from "@/server/actions/transaction";
>
> function AddButton() {
>  return <button onClick={() => createTransaction(data)}>추가</button>;
>  // ↑ 버튼 클릭 → 서버 함수가 실행됨 (API Route 안 만들어도 됨!)
> }
> ```
>
> **왜 Redux/Zustand가 불필요한가**: 기존에는 데이터를 브라우저에서 fetch하고 전역 store에 캐싱해야 했음. 서버 컴포넌트는 요청마다 서버에서 DB를 직접 읽으니 캐싱할 필요가 없음. 데이터 변경은 Server Action 호출 → `revalidatePath("/경로")` 로 해당 페이지를 서버에서 다시 렌더링. 전역 상태가 구조적으로 불필요해짐.

A-2. Next.js & 아키텍처

Q5. SSR, SSG, ISR 차이를 설명해주세요. ⭐⭐⭐ (높은 빈도)

> **모범 답변**:
>
> - **SSR (Server-Side Rendering)**: 요청할 때마다 서버에서 HTML 생성. 항상 최신 데이터. 느릴 수 있음.
> - **SSG (Static Site Generation)**: 빌드 시 HTML 생성. 가장 빠름. 데이터가 바뀌면 재빌드 필요.
> - **ISR (Incremental Static Regeneration)**: SSG + 일정 주기로 재생성. Next.js 고유 기능.
> - **CSR (Client-Side Rendering)**: 빈 HTML 전송 → 브라우저에서 JS로 렌더링. SPA(React 기본 방식).
>
> **이 프로젝트는 SSR**이다. 사용자별 데이터(거래, 예산)가 실시간으로 변하므로 요청마다 서버에서 DB를 조회해 최신 HTML을 생성. SSG/ISR은 "모든 사용자에게 같은 내용"인 블로그 같은 사이트에 적합하고, 개인화된 데이터를 다루는 이 앱에는 부적합.

Q6. Server Component에서 할 수 없는 것은? ⭐⭐ (보통 빈도)

> **모범 답변**: `useState` , `useEffect` 등 React hooks 사용 불가. `onClick` 같은 이벤트 핸들러 불가. 브라우저 API(`window` , `document`) 접근 불가. 즉, **인터랙션이 필요한 것은 전부 Client Component**에서 해야 합니다.

```
// ✅ Server Component (기본값, "use client" 없으면 서버)
// — DB 접근 가능, hooks 불가, 이벤트 불가
async function TransactionsPage() {
    const data = await getTransactions("2025-03"); // 서버에서 DB 직접 조회
    return <TransactionList data={data} />;          // 결과만 클라이언트로
}

// ✅ Client Component ("use client" 선언)
// — hooks 가능, 이벤트 가능, DB 접근 불가
"use client";
function TransactionList({ data }: { data: Transaction[] }) {
    const [filter, setFilter] = useState("");        // useState 가능!
    return <button onClick={() => setFilter("식비")}>필터</button>; // onClick 가능!
}
```

Q7. `revalidatePath` 와 `revalidateTag` 의 차이는? ⭐⭐ (보통 빈도)

> 발음: **리밸리데이트패스** / **리밸리데이트태그** (revalidate = "다시 검증하다")

> [!NOTE] **이것들이 뭐고 어디서 쓰이나?**
>
> Next.js에서 서버 컴포넌트가 데이터를 가져오면, 그 결과를 **캐시**(임시 저장)한다. 데이터가 바뀌었을 때 이 캐시를 "이제 오래됐으니 새로 가져와"라고 알려주는 게 **revalidate**다.
>
> **`revalidatePath("/transactions")`** — 이 프로젝트에서 실제 사용 중:
>
> - 거래를 추가/수정/삭제하는 Server Action 끝에 호출
> - "이 URL 경로의 캐시를 버려라" → 다음에 `/transactions` 방문하면 서버가 DB를 다시 조회
>
> ```
> // src/server/actions/transaction.ts 에서 실제 사용 예
> "use server";
> export async function deleteTransaction(id: string) {
>  await db.delete(transactions).where(eq(transactions.id, id));
> ```

```
    revalidatePath("/transactions"); // ← 여기! 삭제 후 목록 자동 갱신
}
```

revalidateTag — 이 프로젝트에서는 아직 미사용.

- `fetch()` 에 `{ next: { tags: ["user-data"] } }` 태그를 달아두고
- `revalidateTag("user-data")` 로 해당 태그의 캐시만 선별 무효화
- 더 세밀한 제어가 필요할 때 사용하는 고급 기능

> **모범 답변**: `revalidatePath` 는 해당 **경로의 캐시를 무효화**합니다. `revalidateTag` 는 `fetch` 에 태그를 달아두고 **태그 단위로 캐시를 선별 무효화**합니다. 이 프로젝트에서는 CRUD 후 `revalidatePath("/transactions")` 로 목록을 갱신합니다.

A-3. 웹 개발 일반

Q8. REST API vs Server Actions? ⭐⭐ (보통 빈도)

> **모범 답변**: REST API는 URL 기반 엔드포인트를 직접 만들고, 클라이언트가 fetch로 호출합니다. Server Actions는 **서버 함수를 직접 import**해서 호출하며, 타입이 자동 전파되고 별도 API Route가 필요 없습니다. 이 프로젝트에서는 LLM 파싱용 `/api/parse` 하나만 API Route로 두고, 나머지 CRUD는 전부 Server Actions입니다.

> [!NOTE] **이 프로젝트의 Server Actions (CRUD 전체 목록)**

파일	주요 함수	역할
`transaction.ts`	`createTransactions` , `getTransactions` , `updateTransaction` , `deleteTransaction`	거래 CRUD + 월별요약, 카테고리비율, 일별지출, 달력데이터
`budget.ts`	`getBudgets` , `upsertBudget` , `deleteBudget`	예산 CRUD
`account.ts`	`getAccounts` , `createAccount` , `updateAccount` , `deleteAccount`	자산/부채 CRUD
`recurring.ts`	`getRecurringTransactions` , `toggleRecurring` , `deleteRecurring`	고정 거래(구독료 등) 관리
`statistics.ts`	`getYearlyStatistics` , `getMonthlyTrend`	통계 조회
`settings.ts`	`updateCategory` , `deleteCategory` , `reorderCategories`	카테고리 설정
`parse-unified.ts`	`parseUnifiedInput`	LLM 파싱 (거래+자산 통합)
`check-email.ts`	`checkEmailExists`	회원가입 시 이메일 중복 확인

Q9. XSS, CSRF를 어떻게 방지하나요? ⭐⭐ (보통 빈도)

> **모범 답변**:
> - **XSS**: React JSX는 기본적으로 문자열을 **이스케이프**합니다. `dangerouslySetInnerHTML` 을 쓰지 않는 한 안전.
> - **CSRF**: Server Actions는 Next.js가 자동으로 CSRF 토큰을 관리합니다. Better Auth의 `trustedOrigins` 설정으로 허용 도메인을 제한합니다.

Q10. 웹 접근성(a11y) 어떻게 고려했나요? ⭐ (낮은 빈도)

> **모범 답변**: shadcn/ui가 내부적으로 **Radix UI 기반**이라 키보드 내비게이션, aria 속성, 포커스 관리가 기본 내장되어 있습니다. 추가로 모바일 터치 영역을 **최소 44px** 이상으로 보장했고(select `h-11` , switch `h-6 w-11`), `prefers-reduced-motion` 을 존중하는 CSS를 적용했습니다.

Part B. 프로젝트 면접 (이 가계부 앱에 대한 질문)

프로젝트 면접은 **내가 만든 것에 대해** 구체적으로 묻는 질문.

B-1. 프로젝트 개요 & 동기

Q11. 이 프로젝트를 한 줄로 설명해주세요. ⭐⭐⭐ (거의 확실)

> **모범 답변**: "자연어로 입력하면 AI가 자동 분류하는 스마트 가계부"입니다. 예를 들어 "어제 스벅 4500원"이라고 치면 AI가 날짜(어제), 금액(4,500원), 카테고리(카페/음료)를 자동 파싱합니다.

Q12. 왜 이 프로젝트를 만들었나요? ⭐⭐⭐ (거의 확실)

> **모범 답변**: 단순히 **귀찮아서** 시작했습니다. 기존 가계부 앱은 날짜 선택 → 카테고리 선택 → 금액 입력 → 메모 입력... 하나 넣는 데 터치가 7~8번 필요합니다. "스벅 4500"이 한 줄이면 끝나야 한다고 생각했고, LLM이 이걸 실현할 수 있겠다고 판단했습니다. 추가로 N분의 1 정산(더치페이) 기능도 계획 중이고, 풀스택을 직접 경험하기 위해 인증, DB, AI, 네이티브 앱까지 하나의 프로젝트로 연결했습니다.

Q13. 기술 스택을 왜 이렇게 선택했나요? ⭐⭐⭐ (거의 확실)

> **모범 답변**:

선택	이유
Next.js 15	SSR + Server Actions으로 풀스택을 단일 프로젝트로
Drizzle ORM	TypeScript 타입 자동 추론이 뛰어나고 Prisma보다 경량
Better Auth	간단한 이메일/암호 인증에 오버스펙 아닌 선택
shadcn/ui	소스를 직접 수정 가능(터치 영역 등), Radix UI 기반 접근성
Capacitor	기존 웹앱을 그대로 네이티브 앱으로 (코드 1벌 유지)
Tailwind 4	utility-first로 빠른 스타일링, CSS 변수 기반 테마

> [!NOTE] **각 기술 상세 설명 (React 문외한을 위한)**
>
> **Next.js 15 — 왜 15인가?**
>
> Next.js는 React 위에 만들어진 **풀스택 웹 프레임워크**다. React만으로는 서버 렌더링, 라우팅, API 서버 등을 직접 구성해야 하지만, Next.js는 이걸 다 내장하고 있다.
>
> - **최신 버전**: 현재(2025년 3월) 최신 안정 버전은 15.x (15.1, 15.2 등 마이너 업데이트 진행 중)
> - **15 vs 14의 핵심 차이**:
> - React 19 공식 지원 (Server Components, Server Actions가 안정 기능으로 승격)
> - **Turbopack** 안정화 (개발 서버 빌드 도구, 아래 참조)
> - `fetch()` 캐싱 정책이 **기본 no-cache**로 변경 (14에선 기본 cache)

- **PPR (Partial Pre-Rendering)** 실험적 도입 — 한 페이지 안에서 정적 부분은 미리 생성(SSG)하고, 동적 부분만 SSR로 채우는 하이브리드 기술. 아직 실험 단계라 이 프로젝트에서는 미사용
- **왜 15를 선택?**: Server Actions를 production 기능으로 쓰려면 React 19 + Next.js 15가 필요. 14에서도 가능하지만 "experimental" 딱지가 붙어있었음

Drizzle ORM — 이게 뭔데?

ORM(Object-Relational Mapping)은 **SQL을 직접 안 쓰고 JavaScript/TypeScript 코드로 DB를 조작**하게 해주는 도구다.

```
// SQL로 직접 쓰면:
SELECT * FROM transactions WHERE user_id = '123' AND date >= '2025-03-01'

// Drizzle ORM으로 쓰면: (TypeScript, 자동완성 + 타입 안전)
const result = await db.select()
    .from(transactions)
    .where(and(
        eq(transactions.userId, "123"),
        gte(transactions.date, "2025-03-01")
    ));
```

- **Prisma와의 차이**: Prisma는 자체 스키마 언어(`.prisma` 파일)를 사용하고 코드를 생성(generate)해야 함. Drizzle은 **TypeScript 파일이 곧 스키마**이고, SQL에 1:1 대응되어 성능 투명성이 높음
- **이 프로젝트에서**: `src/server/db/schema.ts` 에 테이블 정의 → TypeScript 타입이 자동 추론 → Server Action에서 타입 안전하게 쿼리

Better Auth — 이것도 뭔데? 왜 이걸 골랐는데?

인증(Authentication) 라이브러리다. "사용자가 로그인/회원가입" 기능을 처리한다.

- **경쟁 라이브러리**: NextAuth.js(Auth.js), Clerk, Supabase Auth 등
- **NextAuth.js에 없는 기능?**: 없다. NextAuth.js가 기능적으로 더 풍부(OAuth, Magic Link 등). 하지만 v4→v5 마이그레이션이 대규모 breaking change여서 커뮤니티가 혼란. Better Auth는 v1부터 깔끔하게 시작 + Drizzle ORM 어댑터 자체 내장이라 설정이 훨씬 간단
- **Clerk 대비 장점**: Clerk은 외부 SaaS(유료)이고 데이터가 외부 서버에 저장됨. Better Auth는 **내 DB에 직접 저장**되므로 데이터 통제권이 있음
- **왜 선택?**: 이메일/비밀번호만 필요한 간단한 앱에서 OAuth(구글/깃허브 로그인)까지 오버스펙. Better Auth는 핵심만 제공하면서도 Drizzle과 매끄럽게 연동됨

---

shadcn/ui — 왜 이걸 썼는데?

UI 컴포넌트 라이브러리다. 버튼, 다이얼로그, 드롭다운 등을 처음부터 만들지 않고 가져다 쓸 수 있다.

- **핵심 차이점**: MUI, Ant Design 같은 라이브러리는 `npm install` 로 설치하고 import하는 "패키지"다. shadcn/ui는 **소스 코드를 내 프로젝트에 복사**하는 방식이다.
- **왜 이게 중요?**: 패키지형은 커스터마이즈가 어렵다(버튼 높이를 3px 늘리고 싶은데 CSS override가 안 됨). shadcn/ui는 소스가 내 프로젝트 안에 있으니 **마음대로 수정 가능**
- **이 프로젝트에서**: 모바일 터치 영역 최소 44px 보장을 위해 select, switch 등의 크기를 직접 수정. 이건 패키지형 라이브러리로는 불가능
- **내부적으로 Radix UI 기반**: Radix UI는 **"헤드리스(headless) UI"** 라이브러리. "헤드리스" = 스타일(CSS)이 전혀 없고 **로직만 제공**. 예: Dropdown을 열고 닫는 상태 관리, 키보드 방향키로 항목 이동, 스크린리더용 aria 속성, ESC로 닫기, 포커스 트랩(모달 밖으로 탭 이동 방지) — 이런 복잡한 인터랙션을 직접 구현하면 수백 줄이지만 Radix가 자동 처리. shadcn/ui = Radix의 로직 + Tailwind의 스타일링을 결합한 것

B-2. 핵심 기능 심층 질문

Q14. LLM 파싱의 정확도는 어떤가요? 실패하면? ⭐⭐⭐ (높은 빈도)

모범 답변: 단순 입력("점심 8000원")은 거의 100% 맞습니다. 복잡한 경우를 위해:

1. **재시도 1회** — 파싱 실패 시 자동 재호출
2. **OOD 필터** — "안녕하세요" 같은 의미 없는 입력은 LLM이 거부 응답을 반환
3. **모델 라우팅** — 짧은 입력은 빠른 모델(Fireworks), 100자 이상은 정확한 모델(Kimi 직접 호출)
4. **수동 수정** — 파싱 결과를 사용자가 확인/수정 후 저장하는 워크플로우
5. **새 카테고리 추천** — 매칭 실패 시 `suggestedCategory` 로 새 카테고리 추가 유도

[!NOTE] **OOD = Out-Of-Domain (도메인 외 입력)**

"도메인"이란 이 앱이 다루는 영역 = 가계부/금융 거래. "안녕하세요", "오늘 날씨 어때?", "자바스크립트 코드 짜줘" 같은 건 가계부와 **관련 없는(Out-of-Domain) 입력**이다.

이 프로젝트에서는 시스템 프롬프트에 OOD 규칙을 명시해서, LLM이 관련 없는 입력을 받으면 `{"rejected": true, "reason": "가계부와 관련 없는 입력입니다"}` 같은 거부 JSON을 반환하도록 했다.

OOD 처리 흐름 (코드 기준):

1. 사용자 입력 → LLM에 시스템 프롬프트 + 사용자 입력 전송
2. LLM이 `{"rejected": true}` JSON 반환
3. `parseUnifiedResponse()` 함수가 `rejected` 필드를 감지 → Error throw
4. 프론트에서 "가계부와 관련 없는 입력입니다" 에러 메시지 표시
5. 거래가 생성되지 않음 → 불필요한 데이터 오염 방지

거부 예시: "오늘 날씨 어때?", "자바스크립트 코드 짜줘", "사랑해" **거부 안 하는 예시**: 금액이 포함된 모든 입력, 은행/카드 메시지, 자산/부채 언급

Q15. 은행 메시지 파싱은 어떻게 동작하나요? ⭐⭐ (보통 빈도)

모범 답변: 카카오톡/SMS 알림 메시지를 붙여넣으면 LLM이 자동 감지합니다. **시스템 프롬프트에 은행 메시지 파싱 규칙을 하드코딩**해두었고, LLM이 이 프롬프트를 참조하여 금액, 가맹점, 카드종류를 추출합니다. 여러 줄이면 배치 파싱으로 다건 거래를 한번에 생성합니다.

꼬리 질문: "규격이 다르면 안 되나?" LLM 기반이라 **고정 정규식(regex)이 아니다**. 특정 은행 포맷에 맞춘 게 아니라, LLM이 자연어를 이해하고 "[카카오뱅크] 출금 5,500원 스타벅스"든 "신한체크 17,000 맥도날드"든 **문맥으로 파악**한다. 프롬프트에는 "잔액/한도/할부 정보는 무시하고, 거래 금액+상호명+날짜만 추출하라"는 규칙만 줌. 새로운 은행이 추가돼도 LLM이 범용적으로 파싱 가능.

[!NOTE] **"학습은 어떻게 시켰나요?" — 핵심 개념 정리**

여기서 "학습"은 AI 모델을 추가 훈련(fine-tuning)시킨 게 아니다. 이미 학습 완료된 LLM(Kimi K2.5)에게 **길고 정교한 시스템 프롬프트**를 보내서 행동 규칙을 지시하는 방식이다. 이걸 **프롬프트 엔지니어링**이라 한다.

실제 프로젝트의 시스템 프롬프트(`src/server/llm/prompt.ts`)에 포함된 규칙들:

```
1단계: 의도 판별 — "거래"인지 "자산/부채"인지 먼저 분류
2단계: 거래 파싱 규칙:
  - 날짜 없으면 오늘. "어제"/"그제" 상대 날짜 계산
  - 급여/월급/용돈 → income, 나머지 → expense
  - 금액 변환: "9천"→9000, "1만5천"→15000, "300만원"→3000000
  - 은행 메시지: 잔액/한도/할부 무시, 거래 금액+상호명만 추출
  - "매달"/"구독" 키워드 → isRecurring: true
```

```
카테고리 목록: [사용자의 실제 카테고리 이름들을 동적으로 삽입]
OOD 거부 규칙: 가계부 무관 입력은 rejected: true 반환
```

즉, 모델은 범용 LLM이고 **프롬프트가 가계부 전문가로 만들어주는 것**이다. 카테고리 목록도 사용자별로 DB에서 가져와 프롬프트에 동적으로 삽입한다.

꼬리 질문: "토큰 비용은?" 시스템 프롬프트가 약 100줄(~1,500 토큰) + 사용자 입력(~~50 토큰) = 호출당 약 **2,000 토큰 입력 + 200 토큰 출력**. Kimi K2.5 기준 호출당 약 $0.001~~0.002 수준으로 매우 저렴. 비용 절감 전략: (1) 짧은 입력은 Fireworks(빠르고 저렴), 긴 입력만 Kimi 직접 호출. (2) OOD 필터로 불필요한 호출 차단. (3) fine-tuning 대신 프롬프트 방식이라 모델 학습 비용 0원.

Q16. 다크모드는 어떻게 구현했나요? ⭐⭐ (보통 빈도)

모범 답변: `document.documentElement.classList.toggle("dark")` 로 `html` 에 `.dark` 클래스를 토글합니다. `globals.css` 에서 `:root` 와 `.dark` 각각 CSS 변수를 정의해두면, Tailwind의 `bg-card` , `text-foreground` 같은 클래스가 **자동으로 다크 컬러를 참조**합니다. 사용자 선택은 `localStorage` 에 저장합니다.

Q17. 이미지 파싱(영수증)은 어떻게? ⭐⭐ (보통 빈도)

모범 답변: 사용자가 영수증 사진을 업로드하면, 이미지를 **base64로 인코딩**해서 **Kimi K2.5 Vision API**의 `image_url` 필드로 전송합니다. Kimi의 멀티모달 기능이 영수증의 텍스트와 레이아웃을 인식해 금액, 가맹점, 날짜를 구조화된 JSON으로 반환합니다.

[!IMPORTANT] **이 프로젝트는 OpenAI(GPT)를 사용하지 않는다!**

실제 사용하는 LLM은 **Kimi K2.5** (중국 Moonshot AI사의 모델)이다.

- **Kimi 직접 호출**: `api.moonshot.ai/v1` 엔드포인트 → 정확도 높음
- **Fireworks AI 경유**: 미국 서버에서 같은 Kimi K2.5를 호스팅 → 속도 빠름
- OpenAI **SDK**(라이브러리)를 사용하지만, 이건 HTTP 클라이언트일 뿐. `baseURL` 만 바꿔서 Kimi/Fireworks 서버에 요청을 보내는 것. GPT 모델을 호출하는 게 아님.

```
// 실제 코드 (src/server/llm/client.ts)
const client = new OpenAI({
    apiKey: process.env.KIMI_API_KEY,
    baseURL: "https://api.moonshot.ai/v1",  // ← GPT가 아니라 Kimi 서버!
});
```

B-3. 아키텍처 & 설계 결정

Q18. 전역 상태 관리를 안 쓴 이유는? ⭐⭐ (보통 빈도)

모범 답변: Next.js App Router에서 Server Component가 **요청마다 서버에서 DB를 직접 조회**하기 때문에, 클라이언트에 데이터를 캐싱할 필요가 없습니다. 상태 변경은 Server Action → `revalidatePath` 로 해당 경로만 갱신합니다. 유일하게 `ManualInputProvider` 에서 Context를 써서 입력 다이얼로그 상태를 전역 공유합니다.

Q19. 화면은 어떻게 구성되어 있나요? ⭐⭐ (보통 빈도)

모범 답변: 크게 **인증 영역**과 **대시보드 영역**으로 나뉩니다.

영역	화면	URL	설명
인증	로그인	`/login`	이메일/비밀번호 폼
인증	회원가입	`/register`	이메일 중복확인 포함
대시보드	거래 목록	`/transactions`	**메인 화면**. 월별 거래 목록 + 요약 카드 + 캘린더
대시보드	통계	`/statistics`	카테고리별 파이차트, 월별 추이 라인차트
대시보드	예산	`/budget`	카테고리별 예산 설정 + 사용률 게이지
대시보드	자산/부채	`/assets`	은행 잔액, 카드 미결제, 대출 잔액 관리
대시보드	설정	`/settings`	카테고리 편집, 다크모드, 계정 관리

인증 화면은 **중앙 정렬 카드** 레이아웃, 대시보드는 **하단 탭바 + 상단 헤더 + 플로팅 자연어 입력바** 레이아웃. Next.js의 Route Group `(auth)` , `(dashboard)` 로 URL에 영향 없이 레이아웃만 분리합니다.

Q20. DB 스키마에서 `userId` 를 왜 `text` 로? uuid 아니고? ⭐⭐ (보통 빈도)

모범 답변: Better Auth가 user ID를 `text` 타입으로 생성하기 때문에, FK 일관성을 위해 앱 테이블도 `text("user_id")` 로 맞췄습니다. 카테고리/거래/예산의 id는 `uuid("id").defaultRandom()` 으로 자동 생성합니다.

꼬리 질문: "보안은?"

- **Row-Level Security**: 모든 Server Action에서 세션의 `userId` 와 데이터의 `userId` 를 비교 → 본인 데이터만 접근 가능
- **SQL Injection**: Drizzle ORM이 쿼리를 자동 파라미터화 → 직접 문자열 결합을 하지 않으므로 안전
- **자산 데이터 암호화**: `ENCRYPTION_KEY` 환경변수로 민감 금융 정보를 서버사이드에서 AES 암호화
- **API 키 보호**: LLM API 키, DB URL 등은 서버 환경변수에만 존재. `NEXT_PUBLIC_` 접두사가 없으므로 클라이언트 번들에 포함되지 않음

B-4. 트러블슈팅 & 개선

Q21. 개발하면서 가장 어려웠던 점은? ⭐⭐⭐ (거의 확실)

모범 답변 (예시):

1. **LLM 응답 불안정성**: JSON으로 응답하라고 해도 가끔 마크다운 코드블록으로 감싸거나 추가 텍스트를 덧붙임 → JSON 추출 로직 + 재시도로 해결
2. **SSR에서 Capacitor**: Next.js가 SSR이라 static export 불가 → WebView에서 배포 URL을 로딩하는 방식 채택
3. **다크모드 깜빡임(FOUC)**: SSR HTML이 라이트 → JS 로드 후 다크로 변경 시 깜빡임 → `suppressHydrationWarning` + 인라인 스크립트로 초기 테마 먼저 적용

Q22. 성능 최적화는 어떻게 했나요? ⭐⭐ (보통 빈도)

모범 답변:

- **Turbopack**: 개발 서버 HMR 10배 이상 빠름 (`next dev --turbopack`)
- **optimizePackageImports**: lucide, recharts, motion의 tree-shaking 강화
- **동적 임포트**: 차트, 통계 등 무거운 컴포넌트 지연 로딩
- **RoutePrefetcher**: 탭 전환 전 다음 경로를 미리 프리페치
- **Skeleton Loading**: 모든 페이지에 `loading.tsx` + `<Skeleton>` 컴포넌트
- **immutable 캐시**: 정적 에셋/폰트에 `max-age=31536000` 설정

> [!NOTE] **Turbopack이란?**
> 웹 개발할 때 코드를 수정하면 브라우저에 반영되어야 한다. 이 "코드 → 브라우저" 변환을 해주는 도구를 **번들러(bundler)**라 한다.
> - 기존: **Webpack** (느림, 대형 프로젝트에서 HMR에 수 초 소요)
> - 대안: **Vite** (Svelte/Vue 진영에서 인기, ESM 기반으로 빠름)
> - Next.js 15: **Turbopack** (Rust로 작성, Webpack 대비 10x 빠른 HMR)
>
> `next dev --turbopack` 으로 개발 서버를 실행하면 코드 수정 시 브라우저 반영이 즉각적이다.
>
> **immutable이란?**
> HTTP 캐시 헤더 `Cache-Control: public, max-age=31536000, immutable` 의 일부다.
> - `max-age=31536000` = 1년(365일 × 24h × 3600s) 동안 캐시 유지
> - `immutable` = "이 파일은 절대 안 변한다"는 뜻. 브라우저가 재검증 요청도 안 보냄
> - 폰트, 아이콘 같은 **절대 안 바뀌는 정적 파일**에 적용 → 재방문 시 다운로드 0
> - `next.config.ts` 에서 이 헤더를 설정함

Q23. 보안은 어떻게 고려했나요? ⭐⭐ **(보통 빈도)**

> **모범 답변**:
> - **인증 이중 보호**: Middleware(쿠키 체크) + Dashboard Layout(세션 검증)
> - **Server Actions**: DB 로직이 서버에만 존재, 클라이언트에 노출 안 됨
> - **입력 검증**: LLM 응답에 대한 유효성 검증 (타입, 금액 범위 등)
> - **trustedOrigins**: Better Auth에서 허용 도메인 제한
> - **환경변수 분리**: `.env` 에 API 키, DB URL 관리, 클라이언트 노출 방지
> - **자산 데이터 암호화**: 민감 금융 정보 서버사이드 암호화 (Phase 구현)

B-5. 확장성 & 미래

Q24. 사용자가 늘어나면 어떻게 스케일? ⭐⭐ **(보통 빈도)**

> **모범 답변**: Vercel이 서버리스 함수를 자동 스케일링하므로 서버는 문제없습니다. DB는 Supabase의 connection pooling으로 대응하고, LLM 호출은 비용이 주요 관심사라 **모델 라우팅**(짧은 입력 → 저렴한 모델)으로 비용을 최적화했습니다. 추가로 Drizzle의 인덱스(`userId + date` 복합 인덱스)로 쿼리 성능을 확보했습니다.

Q25. 다음에 추가하고 싶은 기능은? ⭐⭐ **(보통 빈도)**

> **모범 답변**: N분의 1 정산(더치페이), 음성 입력, 은행 API 자동 연동을 계획하고 있습니다. 특히 정산 기능은 DB 스키마(settlements, settlement_members 테이블)까지 설계해둔 상태입니다.

Part C. 압박/심화 질문 (고난도)

면접관이 깊이를 테스트하는 날카로운 질문들.

Q26. "바이브 코딩했다고 하셨는데, 직접 작성한 코드가 있나요?" ⭐⭐⭐ **(핵심)**

> **모범 답변**: AI 도구를 적극 활용한 건 맞습니다. 하지만 **아키텍처 결정, 프롬프트 엔지니어링, 디자인 시스템 설계, DB 스키마 설계**는 직접 했습니다. AI가 생성한 코드도 **코드 리뷰하고 수정하는 과정**을 거쳤습니다.

> [!IMPORTANT] 이 질문은 **반드시 나옵니다**. "다 AI가 했다"가 아니라, **"AI를 도구로 활용하되, 판단과 설계는 내가 했다"** 라는 톤으로 답변하세요.

> [!NOTE] **"아키텍처, 프롬프트, 디자인, DB 스키마를 뭘 어떻게 했는데?" — 구체적으로**
>
> **1. 아키텍처 결정 — 내가 한 것:**
> - Next.js App Router의 Route Group 구조를 `(auth)` / `(dashboard)` 로 나눈 건 레이아웃 분리 목적. AI가 "이렇게 나눠라"고 한 게 아니라, 로그인 화면과 대시보드 화면의 레이아웃이 완전히 달라야 했기 때문에 **직접 구조를 설계**
> - Server Actions를 CRUD 전체에 쓰고 API Route는 LLM 파싱 하나만 두기로 한 판단도 "API Route를 20개 만들 바에 Server Action이 더 깔끔하겠다"는 **트레이드오프 분석 결과**
> - LLM 모델 라우팅 (짧은 입력 → Fireworks, 긴 입력 → Kimi 직접)은 비용과 속도의 **균형점을 실험해서 결정**
>
> **2. 프롬프트 엔지니어링 — 내가 한 것:**
> - 시스템 프롬프트 100줄을 직접 작성하고 **반복 실험으로 튜닝**
> - "점심 김치찌개 9000" → 정확하게 {date: 오늘, category: 식비, amount: 9000} 나오기까지 프롬프트를 수십 번 수정
> - OOD 필터("안녕하세요" 거부), 한국어 금액 변환 규칙("9천"→9000), 은행 메시지 포맷 규칙을 프롬프트에 직접 설계
> - 카테고리 목록을 사용자별로 DB에서 동적 주입하는 구조도 직접 설계
>
> **3. 디자인 시스템 설계 — 내가 한 것:**
> - oklch 컬러 시스템을 채택한 이유: 차트에서 색상 간 시각적 균일성이 sRGB보다 뛰어남. 이 판단은 직접 컬러 팔레트를 비교 테스트한 결과
> - 모바일 터치 영역 최소 44px 보장 정책, `prefers-reduced-motion` 적용 정책도 직접 수립
> - shadcn/ui 컴포넌트의 기본 크기를 프로젝트에 맞게 커스터마이즈 (select 높이, switch 크기 등)
>
> **4. DB 스키마 설계 — 내가 한 것:**
> - 거래-카테고리 관계를 1:N (FK)으로 할지, M:N (중간 테이블)으로 할지 **직접 결정** (1:N 채택)
> - Better Auth의 user ID가 text 타입이라서 앱 테이블도 text로 맞추는 **호환성 판단**
> - `onDelete: cascade` 정책 (사용자 삭제 시 모든 관련 데이터 자동 삭제)도 직접 설정
> - `userId + date` 복합 인덱스로 월별 조회 성능 확보하는 것도 쿼리 패턴을 분석해서 결정

Q27. "이 코드가 이해가 안 되는데 설명해주세요" (라이브 코딩) ⭐⭐⭐

면접관이 프로젝트 코드를 열고 랜덤으로 물어볼 수 있음. **자주 나오는 패턴**:

```
// Server Actions 패턴
"use server";
export async function getTransactions({ month, userId }: Params) {
    const result = await db
        .select({ ... })
        .from(transactions)
        .leftJoin(categories, eq(transactions.categoryId, categories.id))
        .where(and(
            eq(transactions.userId, userId),
            gte(transactions.date, startDate),
            lt(transactions.date, endDate),
        ))
```

```
        .orderBy(desc(transactions.date));
    return result;
}
```

설명 포인트: "use server" 지시자, Drizzle의 select/join/where chain, SQL 조건(and, eq, gte, lt), 날짜 범위 필터, 정렬.

Q28. "React 18/19의 새 기능 뭐 쓰고 있나요?" ⭐⭐

> **모범 답변**: React 19의 Server Components, Server Actions, Suspense 경계를 핵심적으로 사용합니다. `loading.tsx` 가 Next.js의 Suspense boundary 인터페이스이고, Concurrent 렌더링 덕분에 무거운 차트 렌더링 중에도 사용자 입력이 블로킹되지 않습니다.

Q29. "테스트는 어떻게 하고 있나요?" ⭐⭐

> **모범 답변**: Vitest로 유틸 함수 단위 테스트를 작성합니다. E2E 테스트는 아직 미구현인데, Playwright를 도입해 핵심 플로우(로그인 → 거래 입력 → 목록 확인)를 자동화하려 계획 중입니다. 현재는 Vercel의 Preview Deploy로 PR별 수동 QA를 진행합니다.

Q30. "이 프로젝트에서 고치고 싶은 코드가 있나요?" ⭐⭐⭐ **(자기 객관화)**

> **모범 답변**: `TransactionList.tsx` 에서 `editingTx && <TransactionEditSheet open={!!editingTx} />` 처럼 **조건부 렌더링과 open prop이 중복**되는 코드가 있습니다. 이러면 Sheet 닫힘 애니메이션이 끊깁니다. `open` prop만으로 제어하도록 리팩토링하면 애니메이션도 자연스럽고 코드도 깔끔해집니다. (← 튜토리얼 1-1에서 발견한 문제)

Part D. 인프라 & 배포 (DB, CI/CD, 네이티브 앱)

D-1. 데이터베이스

Q31. DB는 뭘 썼고, 왜 그걸 선택했나요? ⭐⭐⭐ **(높은 빈도)**

> **모범 답변**: **PostgreSQL** (via **Supabase**)을 사용합니다.
>
> | 항목 | 내용 |
> |---|---|
> | **DB 엔진** | PostgreSQL 15 — 세계에서 가장 널리 쓰이는 오픈소스 관계형 DB |
> | **호스팅** | Supabase (Free tier) — PostgreSQL을 클라우드로 호스팅해주는 BaaS |
> | **ORM** | Drizzle ORM — TypeScript 타입 자동 추론, SQL에 1:1 대응 |
> | **마이그레이션** | `drizzle-kit push` — 스키마 변경을 DB에 자동 반영 |
> | **접속 방식** | `DATABASE_URL` 환경변수에 Supabase connection string 저장 |
>
> **왜 Supabase?**: (1) Free tier로 시작 가능 (500MB), (2) PostgreSQL을 그대로 제공하므로 vendor lock-in 없음 (다른 PostgreSQL 호스팅으로 언제든 이전 가능), (3) 대시보드에서 테이블/쿼리를 GUI로 확인 가능
>
> **왜 MongoDB가 아닌 PostgreSQL?**: 가계부 데이터는 거래-카테고리-사용자 간 **관계(FK, JOIN)**가 핵심. NoSQL은 관계 표현이 약하고 JOIN이 없어 비효율적. 예산 대비 지출 비율 같은 집계 쿼리도 SQL이 압도적으로 편리.

Q32. DB 테이블 구조를 설명해주세요. ⭐⭐ **(보통 빈도)**

> **모범 답변**: (`src/server/db/schema.ts` 기준)
>
> | 테이블 | 주요 컬럼 | 관계 |
> |---|---|---|
> | `user` | id(text), email, name | Better Auth 자동 관리 |
> | `session` , `account` , `verification` | — | Better Auth 인증 테이블 |
> | `category` | id(uuid), name, type(income/expense), icon, color, sortOrder | user → category (1:N) |
> | `transaction` | id(uuid), amount, description, date, type, memo, isRecurring | user → transaction (1:N), category → transaction (1:N) |
> | `budget` | id(uuid), amount, month | user → budget (1:N), category → budget (1:N) |
> | `account` (자산) | id(uuid), name, type, balance, isHidden | user → account (1:N) |
>
> **핵심 설계 포인트**:
>
> - 모든 테이블에 `userId` FK → `onDelete: cascade` (사용자 삭제 시 모든 데이터 자동 삭제)
> - 회원가입 시 `auth.ts` 의 `user.create.after` hook이 **기본 카테고리 12개를 자동 시딩**
> - `transaction.date` 에 인덱스 → 월별 조회 성능 확보

D-2. 배포 & CI/CD

Q33. 어떻게 배포했나요? ⭐⭐⭐ **(높은 빈도)**

> **모범 답변**: **Vercel**에 배포했습니다.
>
> **Vercel이 뭔데?**: Next.js를 만든 회사의 호스팅 플랫폼. Next.js에 최적화되어 있어 설정 없이 바로 배포 가능.
>
> **배포 과정**:
>
> 1. GitHub 저장소를 Vercel에 연결 (최초 1회)
> 2. `git push` → Vercel이 자동으로 빌드 (`next build`) + 배포
> 3. 배포 URL: `https://household-account-book-tawny.vercel.app`
>
> **환경변수 관리**: `DATABASE_URL` , `KIMI_API_KEY` , `FIREWORKS_API_KEY` , `BETTER_AUTH_SECRET` , `ENCRYPTION_KEY` 등을 Vercel 대시보드의 Environment Variables에 등록. 코드에는 포함되지 않음.

Q34. CI/CD는 어떻게 구성했나요? ⭐⭐ **(보통 빈도)**

> **모범 답변**: **Vercel의 자동 배포 파이프라인**을 사용합니다. 별도의 GitHub Actions나 Jenkins는 없습니다.
>
> | 트리거 | 동작 |
> |---|---|
> | `main` 브랜치에 push/merge | **Production 배포** (실제 서비스 URL) |
> | PR 생성 | **Preview 배포** (PR별 고유 URL 생성 → 리뷰어가 바로 확인 가능) |
> | 빌드 실패 | 자동으로 배포 중단 + 이전 버전 유지 |
>
> **왜 별도 CI/CD를 안 만들었나?**: 개인 프로젝트에서 GitHub Actions + Docker + AWS 같은 파이프라인은 오버엔지니어링. Vercel이 빌드/배포/롤백/Preview를 전부 자동 처리하므로 추가 설정이 불필요.

> **꼬리 질문: "테스트 자동화는?"** 현재 Vitest 단위 테스트 존재. PR 시 Vercel 빌드 과정에서 `next build` 가 타입 체크 + 빌드 검증을 수행. 향후 GitHub Actions에 `vitest run` + Playwright E2E를 추가해 PR 머지 전 자동 테스트 게이트를 만들 계획.

D-3. 네이티브 앱

Q35. 모바일 앱은 어떻게 만들었나요? ⭐⭐ **(보통 빈도)**

> **모범 답변**: **Capacitor**로 네이티브 앱을 만들었습니다.
>
> **Capacitor가 뭔데?**: Ionic팀이 만든 도구. 웹앱을 **네이티브 앱 껍데기(WebView)**로 감싸서 iOS/Android 앱스토어에 올릴 수 있게 해줌. React Native처럼 네이티브 코드를 새로 짜는 게 아니라, **기존 웹앱을 그대로 사용**.
>
> **이 프로젝트 방식**:
>
> ```
> // capacitor.config.ts
> const config: CapacitorConfig = {
>  appId: "com.household.accountbook",
>  appName: "스마트 가계부",
>  server: {
>  url: "https://household-account-book-tawny.vercel.app", // ← Vercel 배포 URL!
>  }
> };
> ```
>
> → 앱을 열면 WebView가 Vercel 배포 URL을 로딩. **웹과 앱이 같은 코드** → 코드 1벌로 웹+iOS+Android 커버.
>
> **장점**: 코드 유지보수 1벌, 웹 업데이트 시 앱도 자동 반영 (앱스토어 심사 불필요) **단점**: 네이티브 성능 제약 (카메라, GPS 등은 Capacitor 플러그인으로 브릿지 필요), 오프라인 미지원

📌 학습 우선순위

```
🔴 반드시 준비 (면접 99% 출제):
   Q11 (한 줄 설명), Q12 (동기), Q13 (기술 선택),
   Q21 (어려웠던 점), Q26 (바이브 코딩 대응),
   Q31 (DB 선택), Q33 (배포)

🟡 높은 확률 (70%+):
   Q1 (VDOM), Q3 (useEffect), Q5 (SSR/SSG),
   Q14 (LLM 정확도), Q30 (고치고 싶은 코드),
   Q34 (CI/CD), Q35 (네이티브 앱)

🟢 준비하면 인상적:
   Q18 (전역 상태 안 쓴 이유), Q22 (성능 최적화),
   Q23 (보안), Q24 (스케일링), Q32 (DB 스키마)
```

🔗 튜토리얼 연계 맵

이 질문집의 답변을 더 깊이 이해하려면 `docs/tutorial/` 의 해당 레슨을 참조하세요.

면접 질문	관련 튜토리얼	상태
Q1. Virtual DOM	1-3 useState (리렌더링 개념)	📝 미작성
Q2. useState vs useRef	2-1 useRef & DOM 접근	📝 미작성
Q3. useEffect	1-4 useEffect	📝 미작성
Q4. 상태 관리	2-3 Context API, 4-4 상태관리 패턴	📝 미작성
Q5. SSR/SSG	3-2 서버 vs 클라이언트 컴포넌트	📝 미작성
Q6. Server Component	3-2 서버 vs 클라이언트 컴포넌트	📝 미작성
Q8. Server Actions	3-4 Server Actions	📝 미작성
Q16. 다크모드	4-1 Tailwind + shadcn/ui	📝 미작성
Q27. 코드 읽기	**1-1 JSX 문법**	✅ 작성 완료
Q30. 코드 개선	**1-1 JSX 코드 리뷰 섹션**	✅ 작성 완료

> [!TIP] 면접까지 시간이 짧다면: 🔴 질문만 달달 외우세요. 시간이 있다면: 튜토리얼 Phase 1 (1-1 ~ 1-6)까지 읽고 프로젝트 코드와 대조하세요.